Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

1. Descrição do sistema

Para o desenvolvimento e modelagem dos dados, o primeiro passo foi compreender o cenário e os requisitos de negócio. A Amazonas tem como objetivo tornar-se o maior e-commerce do Brasil, oferecendo uma vasta gama de produtos, de eletrônicos a vestuário, passando por utensílios domésticos e livros.

O contexto de mercado justifica este objetivo, pois o Brasil possui uma população estimada em 213 milhões de habitantes, dos quais aproximadamente 94,2 milhões realizam compras online com frequência, ao menos uma vez por mês nos principais e-commerces do país. Dados de 2025 reforçam essa tendência, onde 4 em cada 10 brasileiros já realizam compras pela internet, independentemente de serem usuários frequentes de tecnologia.

Esse entendimento de mercado é fundamental para o desenvolvimento de uma solução que não apenas funcione no dia a dia, mas que sustente a demanda em picos sazonais, como Black Friday, Natal e datas comemorativas, momentos em que sistemas mal dimensionados frequentemente falham.

A escolha por um banco de dados não relacional é a mais adequada para este cenário. Para requisitos de alta escalabilidade, flexibilidade de schema e alta disponibilidade, o modelo NoSQL se destaca sobre o relacional, permitindo o uso de técnicas como escalonamento horizontal e sharding. Ao mesmo tempo, o MongoDB (tecnologia adotada neste projeto) garante atomicidade a nível de documento por padrão e suporta transações multi-documento desde a versão 4.0, assegurando a integridade necessária para um ambiente com transações financeiras.

Referências do mercado sustentam a escolha pelo modelo NoSQL: a Amazon utiliza o DynamoDB para processar bilhões de operações diárias. Já o Mercado Livre adotou o MongoDB em partes críticas de sua plataforma pela flexibilidade de schema e escala horizontal. Ambos os casos demonstram que bancos NoSQL são a escolha das maiores plataformas de e-commerce do mundo para workloads de alta escala.

O MongoDB foi escolhido em detrimento do DynamoDB por oferecer maior flexibilidade na modelagem de schemas complexos, suporte nativo a documentos aninhados e uma curva de aprendizado mais acessível para equipes que já trabalham com JSON. O DynamoDB, apesar de sua excelência em escala, exige um design de acesso mais rígido desde o início, o que pode limitar a evolução do schema em um sistema em crescimento acelerado. Por ser uma solução proprietária da AWS, o DynamoDB também apresenta risco de vendor lock-in, limitando opções de migração futura e potencialmente elevando custos operacionais a longo prazo.

Para garantir elasticidade, a solução prevê implantação inicialmente no MongoDB Atlas, que oferece auto-scaling automático de compute e storage, permitindo que o sistema expanda sua capacidade em picos como Black Friday e reduza após o evento, otimizando custos operacionais.

Por fim, embora este documento não aborde em profundidade os temas de segurança e conformidade com a LGPD, esses requisitos foram considerados desde o início da concepção do sistema. Dados sensíveis, como documentos fiscais, informações de pagamento e dados pessoais, não devem ser acessíveis a usuários sem as devidas permissões. Técnicas de hash e criptografia devem estar presentes na camada de aplicação, e certificados SSL são obrigatórios para garantir a segurança na comunicação entre cliente e servidor, conferindo credibilidade e confiança à plataforma Amazonas.

Aluno: Ighor Travassos do Amaral

2. Modelagem de Dados Não-Relacional

2.1. Visão macro: Database e coleções

Ferramenta utilizada: Hacklolade

Database: amazonas_distdb

Coleções

Users: Dados dos clientes.
Schema rígido, mas com os perfis dos usuários centralizados, onde nome, endereço e métodos de pagamento ficam em um único documento, evitando joins (desnormalização).

Products: Catálogo de produtos.
Schema flexível, permitindo a centralização dos produtos, mas comportando categorias e atributos diferentes entre si, sem necessidade de alteração de estrutura. Um notebook e um livro, por exemplo, possuem atributos completamente distintos, e o MongoDB nos permite isso nativamente.

Cart: Carrinho do cliente.
Contém as principais informações necessárias, centralizadas, mas separadas de orders devido ao seu ciclo de vida distinto. Foi adotada a abordagem de expiração automática via TTL index, com janela de 7 dias, não sendo armazenado como histórico permanente no banco. Esta decisão foi tomada para economia de recursos iniciais, mas poderá ser revisada posteriormente, viabilizando análises de carrinhos abandonados e estratégias de recuperação de vendas, por exemplo.

Orders: Pedidos em sistema.
Esta coleção é considerada uma das mais críticas do sistema, armazenando um snapshot completo no momento da compra, garantindo a imutabilidade do histórico financeiro. Note que diversas coleções atreladas à logística e ao financeiro deverão estar integradas posteriormente com os pedidos.

Coupons: Cupons de desconto.
Coleção independente, pois possui regras específicas de negócio, como validade, limite de uso por quantidade de usuários, categorias elegíveis, entre outras.

Reviews: Avaliações dos produtos.
A decisão por manter uma coleção separada de produtos foi de performance, pois um único produto pode ter milhares de avaliações, e embutir tudo isso dentro da coleção de produto comprometeria tanto o tamanho do documento quanto a performance das consultas. Como requisito de negócio, a captura do ID da coleção Orders garante que aquele usuário de fato comprou aquele produto. Posteriormente, regras como avaliações somente de ordens finalizadas e sem reembolso podem ser inseridas diretamente na aplicação, como parte da regra de negócio.

Aluno: Ighor Travassos do Amaral

2.2. Configurações padrão das coleções

Properties (Details)

Capped: False.
Se estivesse habilitado (True), perderíamos os dados mais antigos, que seriam sobrescritos, devido ao limite fixo de bytes por coleção.

Time series: False.
Não aplicável ao contexto, por não se tratar de dados sequenciais/temporais.

Size: Null.
Não aplicável ao contexto, devido ao Capped estar desabilitado.

Max: Null.
Não aplicável ao contexto, devido ao Capped estar desabilitado.

Encryption metadata: False.
Optei por tratar temas de criptografia e segurança no nível da aplicação, não no banco em si, devido à necessidade de alta flexibilidade e independência de versões das ferramentas (disponível somente no MongoDB Enterprise).

Storage engine: WiredTiger.
Devido aos requisitos do sistema, devemos manter os dados armazenados em disco, garantindo persistência mesmo que os servidores reiniciem. A alternativa, In-Memory, que armazena tudo na RAM é extremamente mais rápida, mas pouco consistente. A ideia é posteriormente utilizar o armazenamento em memória para tokens de sessão e chaves de acesso temporárias, otimizando custos e latência.

Validation level: Strict.
Devido à necessidade de mantermos a integridade dos dados, optei por ser mais restritivo com o Strict, pois desta maneira o banco validará todas as operações de inserção e atualização, verificando se violam o schema definido. Qualquer documento que viole o schema é rejeitado antes de entrar no banco, garantindo assim a consistência.

Validation action: Error.
Como estamos trabalhando com Strict, optei pelo Error, retornando uma mensagem de erro para a aplicação, em caso de violação do schema.

Additional properties: True.
Devido à necessidade de alta flexibilidade, optei pelo True, com isso, campos adicionais ao schema serão aceitos, permitindo que o sistema evolua e receba novos campos sem necessidade de alterar o schema. Entretanto, especificamente para campos de endereço e pagamentos, optei pelo False, mantendo alta rigidez para os inputs destes dados.

Collation: True.
Habilitado para definir regras baseadas em idioma.

Aluno: Ighor Travassos do Amaral

Locale: pt.
Definição do português como idioma base para as regras de comparação. Esta decisão acaba afetando as ordenações, buscas e comparações de strings em todas as coleções, garantindo assim que o comportamento seja adequado para o contexto brasileiro. Entretanto, em caso de expansão para outros países, poderá ser utilizado a técnica de sharding por região, semelhante à abordagem da Amazon, por exemplo. Com isso, podemos definir regras de idiomas específicas para cada país.

Strength: 1 — Primary.
Dado o contexto brasileiro, optei pelo Strength 1, ignorando maiúsculas e acentos. Por exemplo, “Jose” e “José” retornariam na busca por “JOSE”.

Case level: False.
Não aplicável ao contexto, devido a seleção do Primary, em Strengh.

Case first: Off.
Não aplicável ao contexto, devido à configuração de PT em locale.

Numeric ordering: True.
Ordenação por ordem numérica, permitindo a ordenação correta em diferentes campos, como ID por exemplo.

Alternate: non-ignorable.
Particularmente, acredito que esta definição é mais adequada ao contexto, para que haja diferenciação de inputs com pontuação diferentes, sejam tratados diferentemente. Entretanto, a seleção de Shifted também seria aplicável. Neste caso, valeria a pena uma discussão mais aprofundada com os stakeholders.

Backwards: False.
Como estamos modelando o sistema para o Brasil, a seleção false permite que o comportamento padrão de leitura, da esquerda para a direita, seja aplicado.

Normalization: False.
Não aplicável ao contexto, devido a tratativa dos inputs dos dados à nível de aplicação, utilizando o enconding UTF-8.

2.3. Detalhes por coleção

Users

Campos e decisões:

- **_id:** objectId, chave primária gerada automaticamente pelo MongoDB.
- **first_name / last_name:** string, mínimo 3 / máximo 50 e 100 caracteres respectivamente. Limites baseados em convenção de mercado. Obrigatórios.
- **email:** string, máximo 254 caracteres (RFC 5321), com validação via regex diretamente no schema do banco como "segunda linha de defesa". Obrigatório.
- **telephones:** array de strings, máximo 16 caracteres por item (padrão E.164 com código de país). Tipo string para preservar zeros à esquerda e o "+" do código de país. Obrigatório, com possibilidade de múltiplos números.

- **birth_date:** string, min/máximo 10 caracteres (formato ISO 8601: YYYY-MM-DD). String em vez de date para evitar conversões de fuso horário e preservar zeros à esquerda. Obrigatório. Observe que este campo se tornará cada vez mais relevante, especialmente após a Lei nº 15.211/2025.

- **address:** objeto embutido com additional properties: false, mantendo a integridade dos inputs. Todos os subcampos são obrigatórios (apenas alguns listados abaixo).
  - ↳ **number:** string para suportar casos como "45-A".
  - ↳ **postal_code:** máximo 10 caracteres, para acomodar formatos internacionais (escalabilidade futura).
  - ↳ **country:** máximo 2 caracteres, padrão ISO 3166-1.

- **tax_id:** string, máximo 20 caracteres. Renomeado de CPF para tax_id para suporte global. O limite de 20 caracteres cobre os principais formatos de documentos. A validação específica por país deverá estar na camada de aplicação. Obrigatório.

- **payment_methods:** array de objetos, sem limite de itens (suporte a múltiplos cartões). Armazena apenas last4 e token, nunca o número real do cartão, em conformidade com PCI-DSS. expires_at com min/máximo 5 caracteres (formato MM/YY). Não obrigatório no schema, permitindo que o usuário se cadastre sem necessariamente adicionar um método de pagamento.

- **status:** bool. Controla ativação/desativação da conta sem deletar o histórico vinculado ao usuário. Obrigatório.

- **created_on:** date. Permite auditorias e análises por período de cadastro. Obrigatório.

- **Índices:** email (unique) e tax_id (unique) para garantir unicidade e otimizar autenticação. created_on para extrações analíticas por período. _id (unique) gerado explicitamente.

- **Sharding:** _id hashed. Garante distribuição uniforme dos cadastros entre os shards, sem hotspots, especialmente relevante em picos como Black Friday, onde novos cadastros são gerados em massa.

Products

Campos e decisões:

- **_id:** objectId, chave primária.

- **name / description:** string, máximo 200 e 5000 caracteres respectivamente. Limites com base em boas práticas de SEO, onde títulos e descrições detalhadas impactam diretamente o posicionamento orgânico. Obrigatórios.

- **code:** string, máximo 50 caracteres. Campo do SKU do produto, utilizado como chave de integração com ERP e sistemas de logística. Obrigatório.

- **type / category / family:** string, máximo 50, 100 e 100 caracteres respectivamente. Taxonomia hierárquica do catálogo, do mais amplo ao mais específico. Base da estratégia de sharding e dos índices de navegação. Todos obrigatórios.

Aluno: Ighor Travassos do Amaral

- **price:** decimal, obrigatório. Decimal em vez de double para garantir precisão exata em valores monetários, evitando imprecisões de ponto flutuante em operações financeiras.
- **stock:** int, mínimo 0, prevenindo estoque negativo no sistema. Obrigatório.
- **images:** array de strings, máximo 2048 caracteres por item (limite padrão para URLs), máximo de 20 itens. Não obrigatório.
- **attributes:** objeto com additional Properties: true. Essa decisão permite que cada produto armazene seus próprios atributos sem necessidade de alteração de schema. Um notebook poderá ter "processador" e "RAM", já uma camiseta, "tamanho" e "material". Não obrigatório.
- **rating_avg:** decimal, mínimo 0 / máximo 5. Campo desnormalizado: a média das avaliações fica diretamente no produto, evitando agregações em tempo real sobre potencialmente milhares de documentos de reviews a cada consulta. Não obrigatório.
- **status:** bool. Caso o produto fique inativo, basta alterar para false, mantendo o histórico de pedidos e avaliações íntegro. Obrigatório.
- **created_on:** date. Permite auditoria e análises temporais, com suporte à ordenação por "Lançamentos", por exemplo. Obrigatório.
- **Índices:** code (unique) para garantir integridade do SKU. category e family para navegação e filtragem no catálogo. rating_avg para ordenação por "Mais bem avaliados".
- **Sharding:** ranged por category, via índice idx_products_category. Agrupa produtos da mesma categoria fisicamente nos mesmos shards, otimizando as consultas de navegação e filtragem por categoria, que são as mais frequentes neste tipo de coleção.

Cart

Campos e decisões:

- **_id:** objectId, chave primária.
- **user_id:** objectId. Referência ao usuário dono do carrinho, utilizada para recuperação no momento do login ou retorno à plataforma. Obrigatório.
- **items:** array de objetos, mínimo 1 item, evitando que carrinhos vazios sejam persistidos no banco. Cada item armazena um snapshot no momento da adição: product_id (referência), name e unit_price (desnormalizados para refletir o estado no momento da inclusão), quantity (int, mínimo 1). Todos os campos do item são obrigatórios.
- **coupon_code:** string, máximo 20 caracteres. Cupom em uso durante o checkout. Não obrigatório.
- **total_price:** decimal. Cache do valor total calculado, evitando recálculo a cada renderização do checkout. Obrigatório.
- **created_on / updated_at:** date. created_on para métricas de abandono de carrinho. updated_at é o campo crítico desta coleção, pois é o campo sobre o qual o TTL index deve operar. Ambos obrigatórios.

- **Índices:** user_id para recuperação do carrinho no login. updated_at com TTL de 604.800 segundos (7 dias), fazendo com que carrinhos inativos sejam excluídos automaticamente pelo MongoDB, sem intervenção da aplicação.

- **Sharding:** não configurado. O ciclo de vida curto dos documentos, excluídos automaticamente a cada 7 dias via TTL, não justifica sharding neste momento, pois o volume máximo retido é naturalmente limitado.

Coupons

Campos e decisões:

- **_id:** objectId, chave primária.

- **coupon_code:** string, máximo 20 caracteres. Obrigatório.

- **discount_type:** string, enum (percentage, fixed). O uso de enum garante integridade dos dados, limitando os valores aceitos aos dois únicos tipos de desconto válidos e protegendo a lógica de cálculo na camada de aplicação. Obrigatório.

- **discount_value / min_order_value:** decimal, mínimo 0. Decimal pela precisão monetária. O mínimo zero impede descontos ou valores mínimos negativos. Ambos obrigatórios.

- **max_uses / uses_count:** int. max_uses define o limite de utilizações do cupom, uses_count é o contador incremental. Campos utilizados em conjunto para validação de disponibilidade do cupom antes da aplicação. Ambos obrigatórios.

- **expires_at:** date. Validade do cupom. Diferentemente do TTL do cart, aqui a expiração não exclui o documento, apenas sinaliza para a aplicação que o cupom não pode mais ser utilizado, preservando o histórico. Obrigatório.

- **created_on:** date. Para auditoria e análises de campanhas. Obrigatório.

- **applicable_to:** objeto com additional Properties: false e dois arrays internos: products_category e products_code. Os arrays permitem que um único cupom seja válido para múltiplas categorias e múltiplos produtos simultaneamente. A estrutura fechada foi definida intencionalmente, pois os critérios de aplicabilidade foram bem definidos. Obrigatório.

- **Índices:** coupon_code (unique) para evitar conflitos e otimizar buscas por código. expires_at para consultas analíticas e de BI sobre campanhas encerradas.

- **Sharding:** não configurado. Trata-se de uma coleção de volume reduzido e com acessos concentrados por código específico, podendo operar sem problemas no shard primário.

Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

Reviews

Campos e decisões:

- **_id:** objectId, chave primária.
- **product_id / user_id / order_id:** objectId. user_id captura o usuário autenticado que realizou a avaliação. order_id garante rastreabilidade, ou seja, apenas usuários que de fato compraram o produto podem avaliá-lo. product_id registra o produto avaliado. Todos obrigatórios.
- **rating:** int, mínimo 1 / máximo 5. Obrigatório.
- **title / description:** string, máximo 100 e 1000 caracteres respectivamente. O limite de 1000 na descrição permite avaliações detalhadas, mas controlado para evitar uso indevido da coleção como “repositório de texto livre”. Ambos não obrigatórios, permitindo que uma avaliação composta apenas pela nota seja válida.
- **helpful_votes:** int, mínimo 0. Contador desnormalizado para ordenar avaliações por "utilidade", conforme votação dos demais usuários da plataforma, sem necessidade de agregações em tempo real. Não obrigatório.
- **created_on:** date. Permite ordenação cronológica das avaliações. Obrigatório.
- **Índices:** product_id para carregamento eficiente das avaliações na página do produto. user_id para que o usuário possa consultar rapidamente suas próprias avaliações.
- **Sharding:** hashed por product_id, via índice idx_reviews_product_id. Distribui avaliações de diferentes produtos uniformemente entre os shards, evitando que produtos com alto volume de comentários (best-sellers, lançamentos) sobrecarreguem um único nó.

Orders

Campos e decisões:

- **_id:** objectId, chave primária. Utilizado também como identificador do pedido visível ao usuário.
- **user_id:** objectId. Referência ao usuário autenticado no momento da criação do pedido. Obrigatório.
- **address_shipping:** objeto com snapshot completo do endereço de entrega no momento do checkout. A mesma estrutura base existe em users, mas é copiada intencionalmente, permitindo que o usuário altere seu endereço de cadastro no futuro sem afetar o histórico de pedidos já realizados. Additional Properties: false. Todos os subcampos obrigatórios.
- **payment**: objeto com method (enum: credit_card, debit_card, pix, ticket, other), status (enum: pending, approved, refused, refunded), token e paid_at. Os enums protegem a integridade dos dados financeiros. O token deve ser gerado pelo gateway de pagamento, nunca armazenando dados reais do cartão. paid_at não é obrigatório, pois pode não ter sido processado no momento da criação do pedido. method, status e token são obrigatórios.
- **coupon_code / discount_amount:** string e decimal. Registro histórico do cupom utilizado e do valor de desconto aplicado, essenciais para relatórios financeiros e análises de campanhas de marketing. Não obrigatórios.

- **total_price:** decimal, mínimo 0. Valor final exato no momento da compra, imutável após confirmação. Decimal para conformidade contábil. Obrigatório.
- **tracking_code:** string, máximo 100 caracteres. Gerado pela transportadora após o despacho do pedido. Não existe no momento da criação, portanto não obrigatório.
- **status:** string, enum (pending, confirmed, shipped, delivered, cancelled). Controla o ciclo de vida do pedido e impede estados inválidos. Obrigatório.
- **items:** array de objetos. Snapshot dos produtos no momento da compra: product_id, name e unit_price (desnormalizados, alterações futuras no catálogo não afetam o histórico financeiro), quantity (int, mínimo 1). Todos os campos do item são obrigatórios.
- **created_on / updated_at:** date. created_on para data de criação do pedido. updated_at para registrar cada transição de status. Ambos obrigatórios.
- **Índices:** user_id para o histórico de pedidos do cliente. _id (unique) para consulta direta de um pedido específico, via índice idx_orders_id. items.product_id para cruzamentos analíticos de BI.
- **Sharding:** ranged por _id, via índice idx_orders_id (unique). A escolha pelo _id como shard key garante unicidade global dos pedidos entre os shards (parâmetro unique: true habilitado no script) e distribui novos pedidos de forma sequencial e controlada. Em picos como Black Friday, onde o volume de criação de pedidos é massivo, essa estratégia evita que todos os writes sejam direcionados a um único shard.

3. Plano de Escalabilidade

3.1. Estratégia de Particionamento (Sharding)

O sharding é a principal estratégia de escalabilidade horizontal adotada neste projeto. Ele permite que os dados sejam distribuídos fisicamente entre múltiplos nós, garantindo que nenhum servidor único se torne o gargalo da operação, especialmente em picos sazonais como Black Friday e Natal, onde o volume de leituras, escritas e acessos simultâneos crescem de forma exponencial.

Segue abaixo a visão consolidada das decisões de sharding por coleção (todas refletidas no script MongoDB):

Users: hashed por _id

O _id hashed garante distribuição uniforme dos documentos de usuários entre os shards, sem agrupamento intencional. Acredito que é a escolha correta para esta coleção, pois o padrão de acesso mais frequente é a busca por um usuário específico (por _id, email ou tax_id), e não por intervalos de valores. Em períodos de alto cadastro, como promoções, a distribuição uniforme evita hotspots em um único shard.

Products: ranged por category, via índice idx_products_category

A estratégia ranged foi escolhida intencionalmente para esta coleção, pois agrupa produtos da mesma categoria fisicamente no mesmo shard. O padrão de acesso mais frequente ao catálogo de produtos é a navegação e filtragem por categoria (comportamento típico de qualquer e-commerce). Manter produtos correlatos fisicamente próximos reduz a necessidade de scatter-gather queries entre shards, otimizando diretamente a experiência de navegação do usuário final.

Aluno: Ighor Travassos do Amaral

Reviews: hashed por product_id, via índice idx_reviews_product_id

A estratégia hashed foi adotada para distribuir avaliações de forma uniforme entre os shards, evitando que produtos com altíssimo volume de avaliações, best-sellers ou lançamentos, concentrem toda a carga em um único nó. Como o volume de avaliações por produto é imprevisível e potencialmente muito assimétrico, o hashed é mais seguro que o ranged neste contexto.

Orders: ranged por _id, via índice idx_orders_id (unique: true)

O _id como shard key com estratégia ranged e unicidade garante que cada pedido tenha um identificador único e rastreável em todo o cluster. A opção unique: true no script reforça a integridade referencial dos pedidos entre os shards. Em picos de alta criação de pedidos, a distribuição sequencial por _id direciona novos documentos de forma controlada entre os shards disponíveis.

Cart, Coupons: sem sharding

O cart possui ciclo de vida curto e controlado pelo TTL index de 7 dias, o que limita naturalmente seu volume máximo retido no banco. O coupons é uma coleção com volume de dados reduzido e com padrão de acesso concentrado por código específico. Ambas operam adequadamente no shard primário, sem necessidade de distribuição horizontal neste momento.

3.2. Estratégia de Replicação

Para garantir alta disponibilidade e tolerância a falhas, a solução deve adotar o modelo de Replica Set do MongoDB, composto por três nós: um nó primário, responsável por receber todas as operações de escrita, e dois nós secundários, que mantêm cópias sincronizadas dos dados em tempo real.

Em caso de falha do nó primário, o MongoDB realiza automaticamente a eleição de um novo primário entre os secundários disponíveis, sem intervenção manual. Isso garante continuidade operacional mesmo em cenários de falha de infraestrutura, o que é crítico para um e-commerce onde a indisponibilidade representa perda direta de receita.

Coleções para replicação

Todas as coleções do banco devem ser replicadas integralmente pelo Replica Set. Isso inclui users, products, cart, coupons, reviews e orders. Não há distinção de política de replicação entre as coleções neste modelo. A replicação é total e contínua, garantindo que qualquer nó secundário possa assumir o papel de primário com o estado mais recente dos dados.

O foco da replicação é, portanto, a disponibilidade e a durabilidade dos dados, enquanto o sharding endereça a distribuição de carga e o crescimento horizontal.

Leitura em nós secundários

Por padrão, todas as operações de leitura devem ser direcionadas ao nó primário, garantindo consistência imediata. Entretanto, para coleções de alto volume de leitura, como products e reviews, a configuração secondary Preferred pode ser ativada como estratégia de desafogo do nó primário. Entretanto, antes de ser alterado, é interessante o alinhamento junto aos stakeholders, com base em thresholds percentuais de utilização (definidos previamente), e ativada de forma elástica conforme a demanda. Conceitualmente, trata-se de escalabilidade horizontal de leitura, sem necessidade de novos shards.

Aluno: Ighor Travassos do Amaral

3.3. Crescimento de Dados

O crescimento de dados é tratado por três mecanismos já presentes no script MongoDB:

O primeiro é o sharding, que permite a adição de novos shards ao cluster conforme o volume de dados cresce, sem necessidade de reestruturação do schema ou interrupção do serviço. As coleções com sharding configurado (users, products, reviews, orders) expandem horizontalmente de forma transparente para a aplicação.

O segundo é o TTL index sobre cart.updated_at, com janela de 604.800 segundos (7 dias). Carrinhos inativos são excluídos automaticamente pelo MongoDB, sem intervenção da aplicação, controlando o crescimento desta coleção de forma autônoma e previsível.

O terceiro são os índices, que garantem que o crescimento do volume de dados não traga gargalo de performance nas consultas mais frequentes. À medida que o volume cresce, as consultas continuam operando sobre estruturas otimizadas, evitando full collection scans.

3.4. Estratégia de Particionamento (Sharding)

Foi adotado neste projeto o WiredTiger storage engine como abordagem para alta concorrência, decisão descrita na seção 2.2. O MongoDB lida nativamente com este cenário por meio deste engine, que utiliza controle de concorrência em nível de documento, permitindo que múltiplas operações ocorram simultaneamente sem bloqueio global da coleção.

O sharding complementa essa capacidade ao distribuir a carga de escritas e leituras entre múltiplos nós, garantindo que picos de concorrência não sobrecarreguem um único servidor. Como camada adicional, foi pensado, como próximo passo do projeto, o uso de Redis como cache externo, operando em duas frentes distintas e em instâncias separadas: uma dedicada ao cache das principais informações da aplicação, como catálogo de produtos e dados de sessão frequentemente consultados, e outra dedicada ao armazenamento de tokens de autenticação e chaves de acesso temporárias. Esta separação garante isolamento entre os dois contextos de uso, otimizando tanto a latência de leitura quanto a gestão do ciclo de vida dos tokens.

3.5. Implementação em Nuvem

A solução foi projetada e documentada considerando o MongoDB Atlas como plataforma de implantação, com suporte nativo a sharding, replica sets e auto-scaling de compute e storage, conforme descrito na seção 1. Entretanto, o deploy no Atlas não foi realizado até o momento de finalização deste trabalho. Consequentemente, o script MongoDB entregue não foi validado em ambiente real, e alguns ajustes manuais poderão ser necessários no momento da implementação, como exemplo, o tratamento de índices gerados pelo Hackolade possui limitações identificadas na exportação do script para ambientes sharded. A arquitetura, contudo, foi desenhada para funcionar corretamente sobre esta plataforma.

Por fim, deixo o repositório do GitHub para acompanhamento do projeto em suas próximas etapas: [https://github.com/i-amaral/amazonas-distdb].

Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

4. Imagens

Figura 1 - ER Diagram

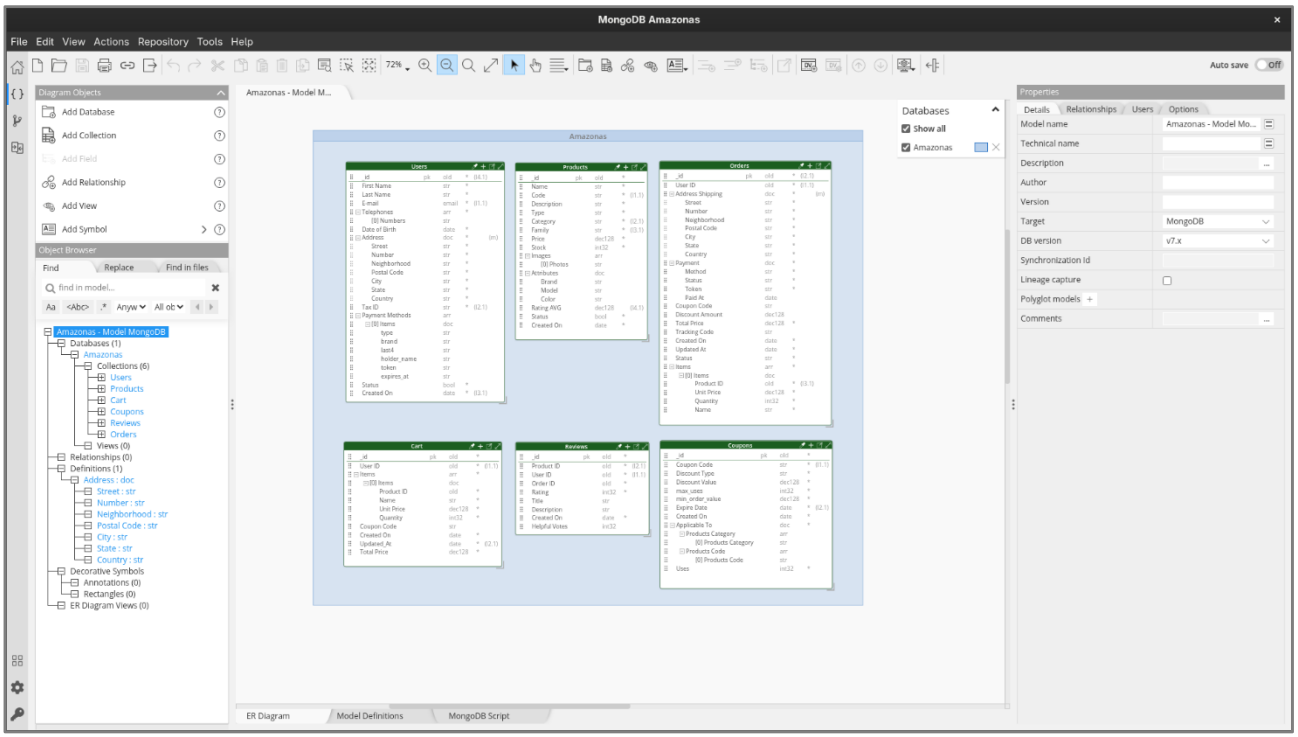
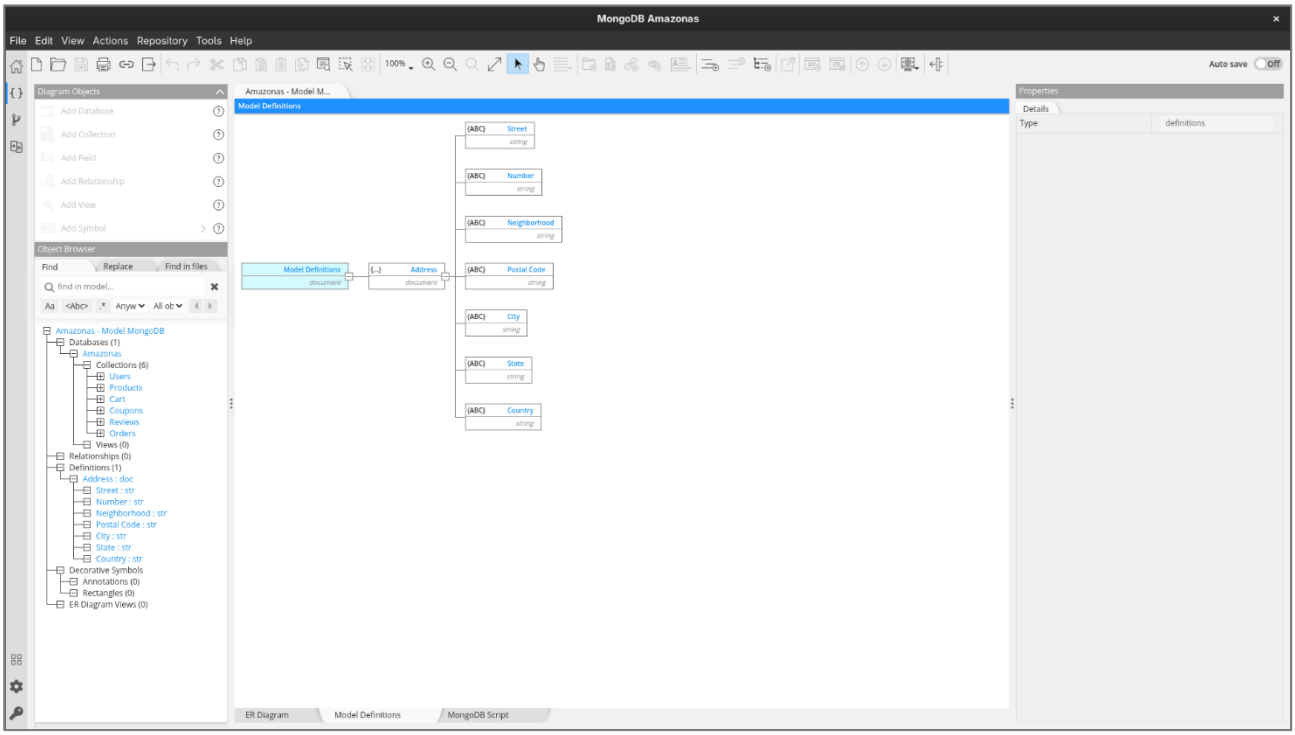


Figura 2 - Model Definitions



Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

Figura 3 - MongoDB Script

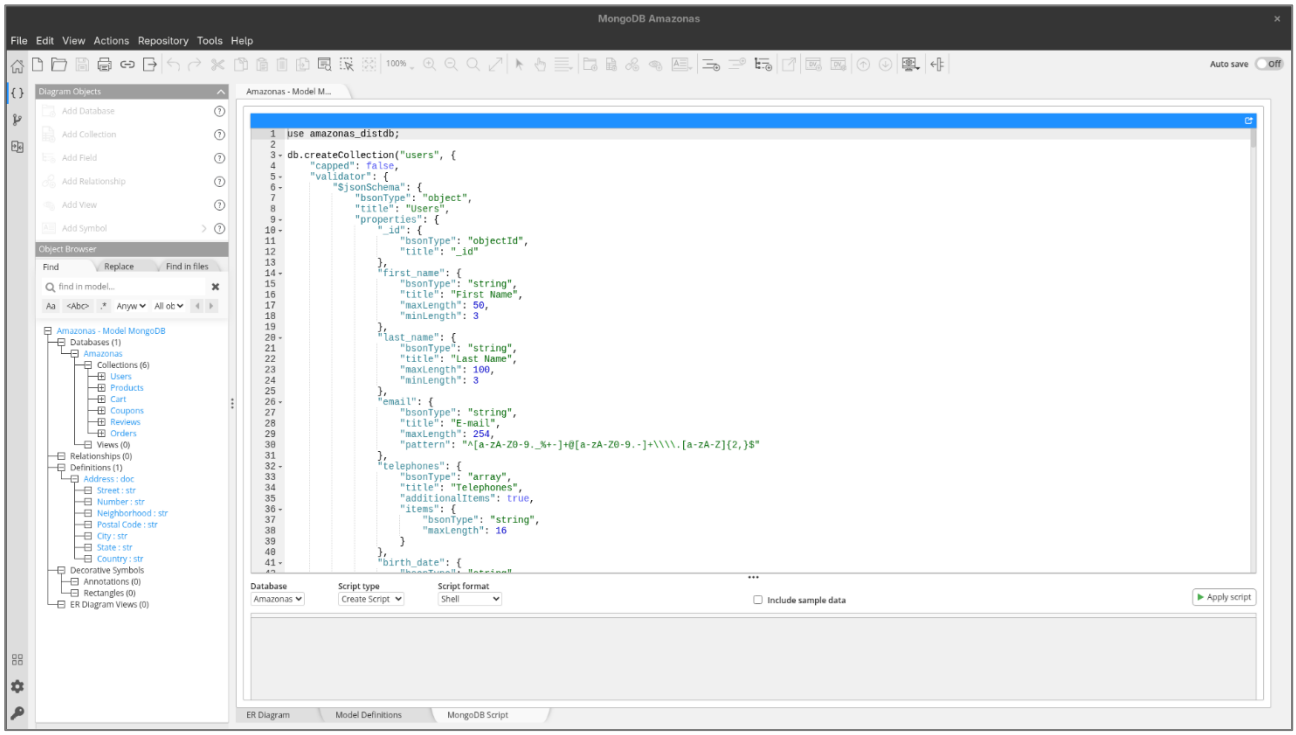
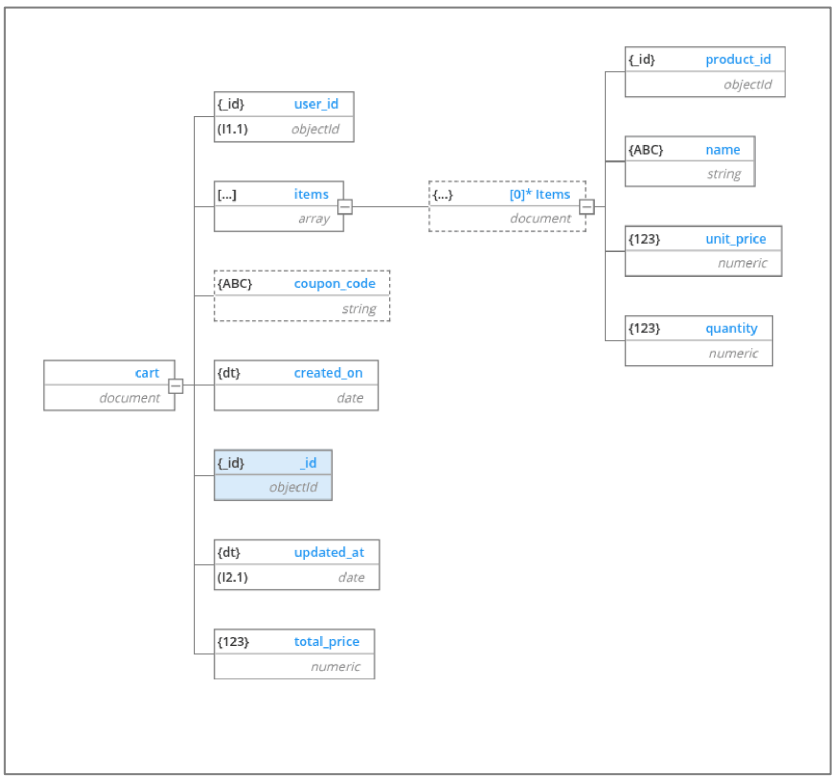


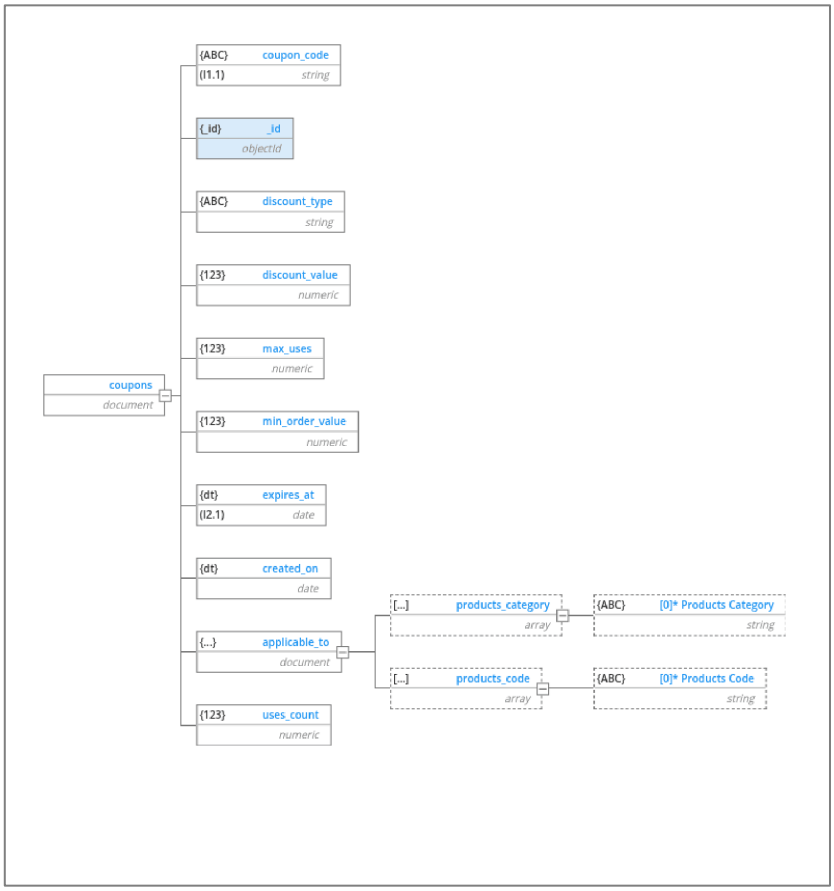
Figura 4 - Model Collection Cart



Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

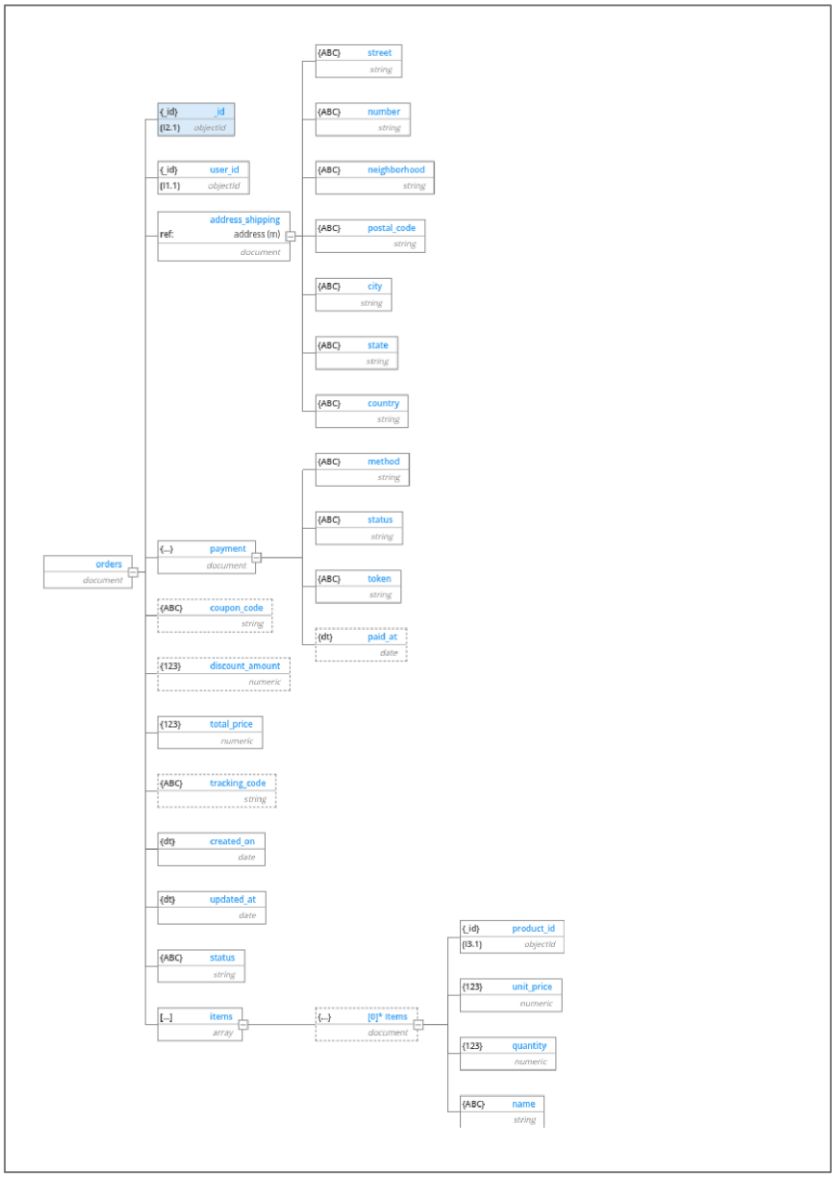
Figura 5 - Model Collection Coupons



Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

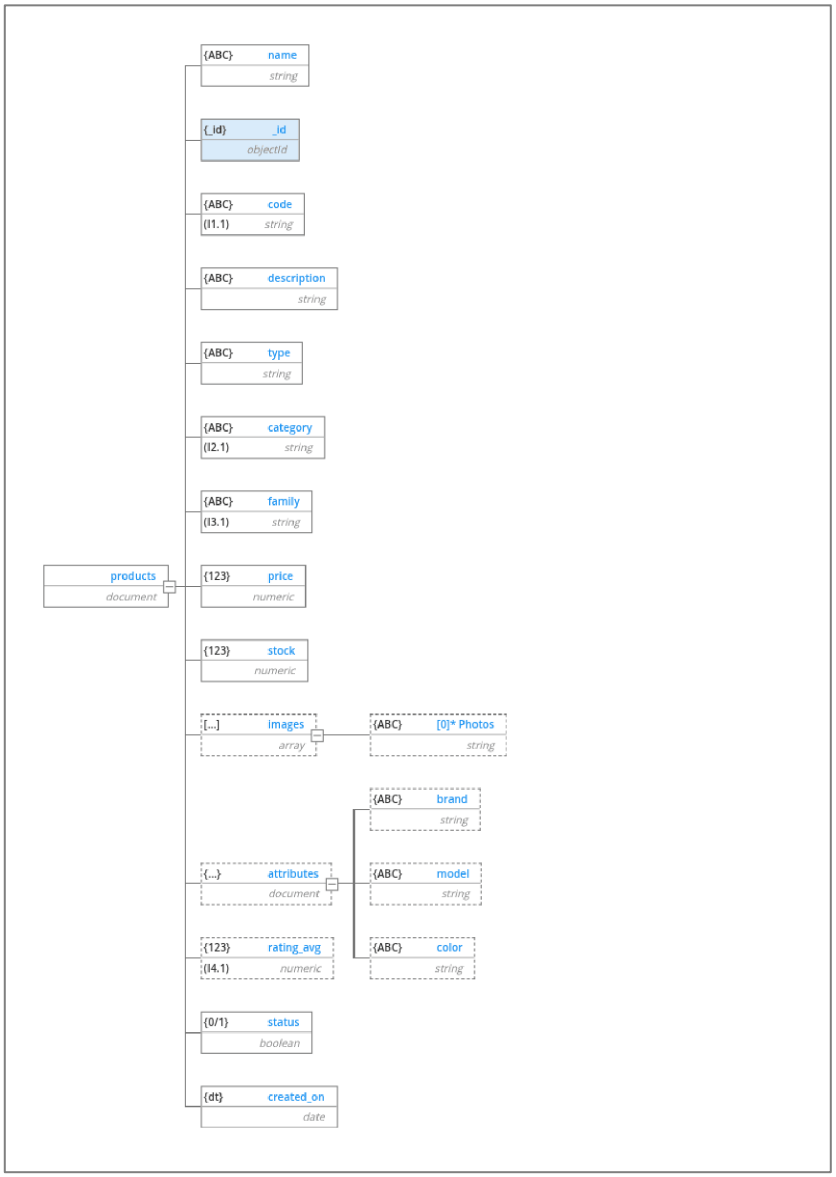
Figura 6 - Model Collection Orders



Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

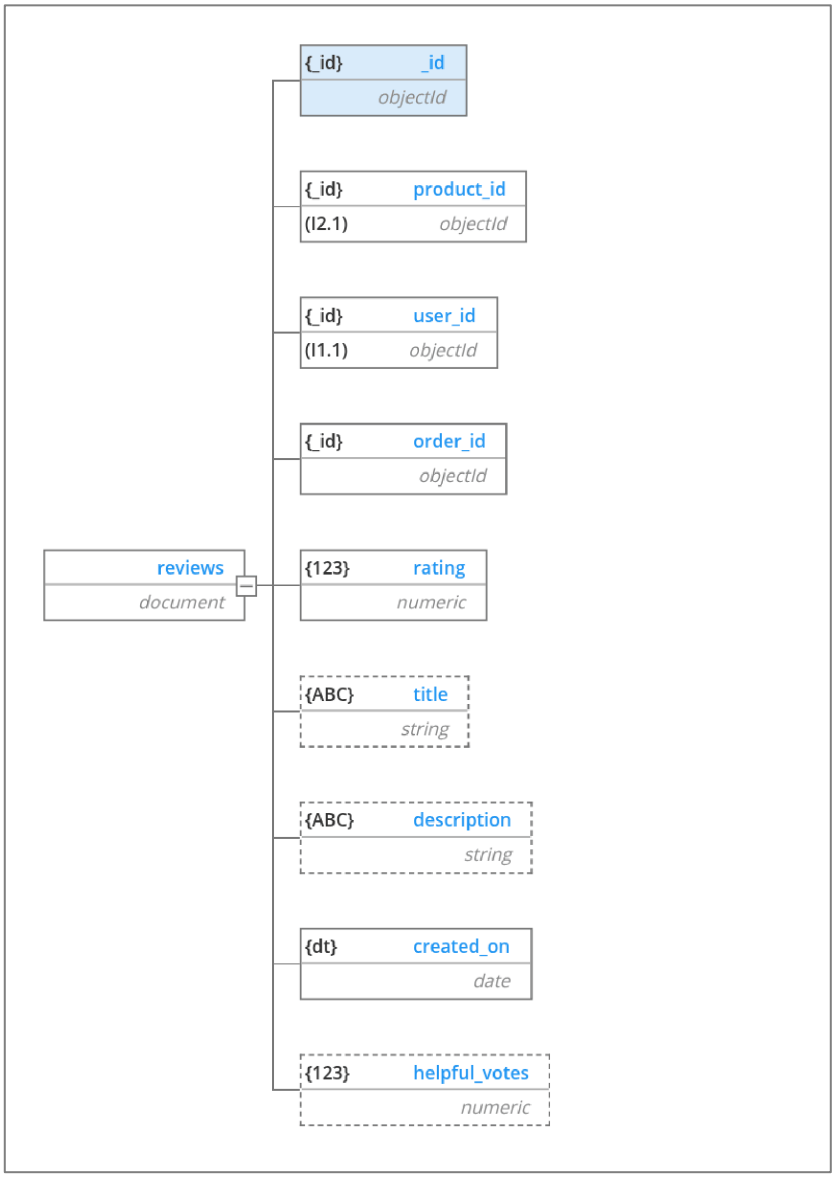
Figura 7 - Model Collection Products



Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

Figura 8 - Model Collection Reviews



Desafio Final – Arquiteto de Dados

Aluno: Ighor Travassos do Amaral

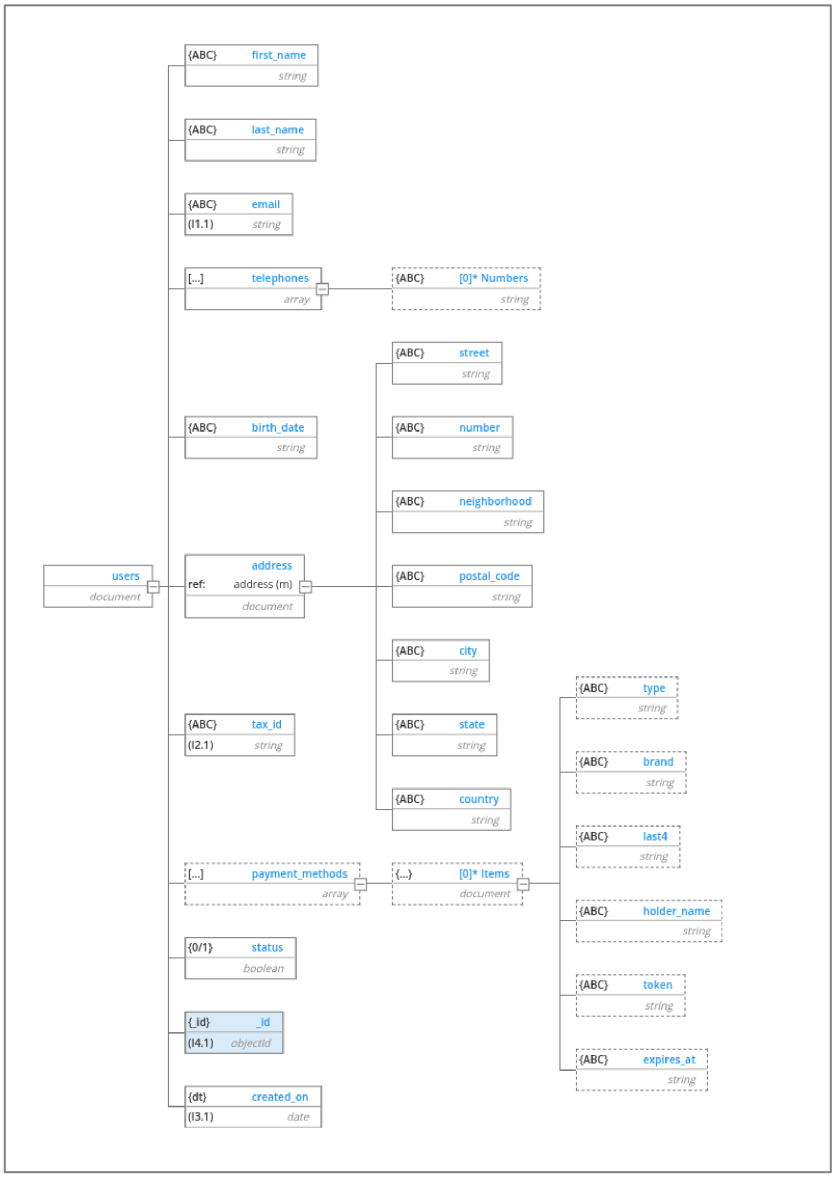



Figura 9 - Model Collection Users